

NumPy Neural Network

Matematyka, kod i eksperyment: opracowanie projektu od podstaw

Streszczenie

Projekt uczy od podstaw niewielką sieć NumPy przypisywać punkt do jednej z dziesięciu klas. Wyjaśniamy, skąd pochodzą dane, co liczy każda warstwa i funkcja straty, w jaki sposób pochodne biegną wstecz oraz gdzie dokładnie zmieniają się wagi. Każdy ważny wzór ma obok wycinek *rzeczywistego* kodu, sprawdzenie wymiarów lub przykład liczbowy. Matematyka zaczyna się od pojedynczej obserwacji i znanej z analizy reguły łańcuchowej; dopiero potem rozszerzamy rachunek na macierze i partie. Na końcu są wyniki uruchomienia, audyt, ćwiczenia oraz pełny kod analizowanej rewizji.

Jak korzystać z dokumentu

Jeśli poprzednio znałeś regresję liniową, możesz myśleć o jednym neuronie jako o jej wyrażeniu $x_1w_1 + \dots + x_dw_d + b$. Nowe elementy to funkcja aktywacji, wiele ocen klas, ich zamiana na udziały oraz przekazywanie pochodnych przez obie warstwy. Zakładam znajomość podstaw pochodnej i mnożenia macierzy na poziomie pierwszego roku WNE; wszystkie nowe pojęcia ML objaśniam przy kodzie. Czytaj po kolei do sekcji o propagacji wstecznej. Dalsze działy możesz traktować jako osobne tematy do powrotu po eksperymencie z kodem. Zapis $X \in \mathbb{R}^{B \times d}$ oznacza po prostu tablicę o B wierszach i d kolumnach. Litera B zmienia się wraz z rozmiarem partii; nie jest kolejnym parametrem sieci.

Spis treści

1 Mapa projektu i pytanie, na które odpowiada sieć	4
1.1 Od punktu do numeru klasy	4
1.2 Pięć pytań prowadzących przez repozytorium	4
2 Notacja i wymiary: krótka mapa tablic	4
3 Dane syntetyczne: klasy mają środki i szum	5
3.1 Najpierw prosty przykład, potem rozkład	5
3.2 Sklejenie, wspólne tasowanie i podział	5
4 Etykieta jako liczba i jako wektor one-hot	6
5 Neuron: ważona suma, bias i przejście do macierzy	6
5.1 Jedna obserwacja i jeden neuron	6
5.2 Po co dwie warstwy?	7
6 ReLU: nieliniowość i bramka dla gradientu	7
7 Początkowe wagi i architektura TwoLayerNet	7
8 Przebieg w przód: od tablicy cech do wyników klas	8
9 Softmax: co oznacza wynik klasyfikatora	9
9.1 Od logitów do udziałów	9
9.2 Udział a pewność modelu	9
10 Entropia krzyżowa: za co model dostaje karę	9
10.1 Jedna etykieta, jeden logarytm	9
10.2 Dlaczego logarytm: jedna intuicja probabilistyczna	10
11 Propagacja wsteczna: co właściwie ma policzyć?	10
12 Krok wstecz I: gradient po logitach	11
12.1 Dwie klasy: rachunek pochodnych bez wielkich sum	11
12.2 Wiele klas i średnia po partii	11
13 Krok wstecz II: wagi i biasy drugiej warstwy	12
13.1 Miniaturowa partia: po co jest suma?	12
14 Krok wstecz III: sygnał dla neuronów ukrytych	12
15 Krok wstecz IV: ReLU i pierwsza warstwa	13
15.1 Podsumowanie pochodnych i wymiarów	13
16 Jeden pełny krok uczenia, policzony na liczbach	14
17 Dwa punkty naraz: skąd biorą się sumy i transpozycje	14
17.1 Sygnały mogą walczyć ze sobą	15
17.2 Bramka ReLU w partii i sens $\mathbf{x} \cdot \mathbf{T}$	15
18 Uczenie na partiach: epoka, krok i aktualizacja	15
18.1 Dlaczego gradient liczymy na 32 punktach?	15
18.2 Tasowanie i wycinanie partii	16
18.3 Gdzie naprawdę zmieniają się parametry?	16
19 Predykcja i dostęp do parametrów	17
20 Sprawdzenie gradientu: niezależny pomiar pochodnej	17
20.1 Jedna waga, dwa dodatkowe przebiegi	17

20.2	Błąd względny i jego ograniczenia	18
20.3	Kontrola jednej rzeczywistej wagi na liczbach	19
21	Jak main składa eksperyment w całość	19
22	Wizualizacja i interpretacja rzeczywistego przebiegu	20
22.1	Co zapisuje visualize.py?	20
22.2	Wyniki z analizowanej rewizji	20
23	Dlaczego mała różnica w kodzie straty ma znaczenie?	21
24	Jaki problem probabilistyczny tworzy generator?	22
24.1	Nakładanie się klas i poziom szumu	22
24.2	Czy sieć musi mieć ReLU dla tego generatora?	22
24.3	Dokładność, entropia i kalibracja odpowiadają na różne pytania	22
25	Skala danych, pochodne i zachowanie uczenia	23
25.1	Dlaczego wielkość wejść ma znaczenie	23
25.2	Czego nie przenosić bezpośrednio z regresji liniowej	23
26	Audyt kodu: co działa dobrze i gdzie są granice	23
26.1	Decyzje trafne w projekcie edukacyjnym	23
26.2	Uproszczenia i ograniczenia do świadomego zbadania	23
27	Co badać dalej w samym projekcie	24
27.1	Trzy kontrolowane zmiany	24
27.2	Most do znanej regresji liniowej	24
27.3	Regularizacja jako możliwy dalszy krok	25
28	Pytania kontrolne i ćwiczenia	25
28.1	Wyjaśnij kod bez patrzenia na wzory	25
28.2	Zrób krótki rachunek	25
28.3	Praktyczny test pełnego zrozumienia	26
29	Słownik potrzebny do powrotu do kodu	26
30	Końcowa synteza	26
31	Niezmieniony kod źródłowy	27

1 Mapa projektu i pytanie, na które odpowiada sieć

1.1 Od punktu do numeru klasy

Wyobraź sobie obiekt opisany trzydziestoma liczbami. Chcemy przypisać mu jedną z dziesięciu etykiet $0, \dots, 9$. Program buduje dwuwarstwową sieć (dwie tablice wag) z 64 neuronami w warstwie ukrytej. Na wejściu ma 30 liczb, na wyjściu 10 *udziałów* sumujących się do 1. Największy udział wyznacza przewidywaną klasę. Trening wielokrotnie dostosowuje wagi tak, by udział *prawdziwej* klasy rósł.

$$\text{punkt } x \xrightarrow{W_1, b_1, \text{ReLU}} 64 \text{ aktywacje} \xrightarrow{W_2, b_2, \text{softmax}} (p_0, \dots, p_9). \quad (1.1)$$

Model nazywa się `TwoLayerNet`: dwie są warstwy z *uczonymi* wagami; warstwa wejściowa bywa liczona osobno przy innych konwencjach. Ten dokument używa konwencji z kodu. Dane są syntetyczne: w tym ćwiczeniu nie ma obrazów, danych klientów ani ogólnego problemu rozpoznawania rzeczywistych obiektów.

1.2 Pięć pytań prowadzących przez repozytorium

1. *Skąd pochodzą punkty?* `run_experiments.py` generuje je, tasuje i dzieli.
2. *Jak liczymy wynik?* `model.py` i `layers.py` implementują `forward`, `ReLU` i `softmax`.
3. *Co znaczy dobry wynik?* `cross_entropy` ocenia udział przypisany poprawnej klasie.
4. *Jak poprawić wagi?* `backward` zwraca pochodne, a `train.py` wykonuje aktualizacje.
5. *Skąd wiemy, że działa?* `gradient_check`, walidacja i `visualize.py` dostarczają różnych, ograniczonych kontroli.

Pusty `__init__.py` służy strukturze pakietu. Program uruchamia się z katalogu głównego repozytorium jako `python -m phase0.projects.numpy_neural_net.run_experiments`; importy zaczynające się od kropki odwołują się do sąsiednich modułów pakietu. Dokładnie tę rewizję 32c0899c48ad wykorzystano w przykładach i wynikach.

2 Notacja i wymiary: krótka mapa tablic

Najpierw jedna obserwacja $x = (x_1, \dots, x_d)$: $d = 30$ cech. $h = 64$ oznacza liczbę neuronów ukrytych, $C = 10$ liczbę klas. Litera i oznacza numer obserwacji, j wejście, r neuron ukryty, k klasę. Dla wielu obserwacji zapisujemy ich wektory jako wiersze tablicy X . Na treningu zwykle $B = 32$; w prostym przykładzie ręcznym będziemy używać $d = h = C = 2$ i $B = 1$. Zmiana rozmiarów przykładu nie zmienia algorytmu.

Obiekt	Kształt w kodzie	Co oznaczają osie?
<code>X_batch</code>	$(B, 30)$	obserwacja, cecha
<code>w1, b1</code>	$(30, 64), (64,)$	cecha do neuronu, osobny bias neuronu
<code>z1, a1</code>	$(B, 64)$	obserwacja, neuron przed / po ReLU
<code>w2, b2</code>	$(64, 10), (10,)$	neuron do klasy, bias klasy
<code>z2, probs</code>	$(B, 10)$	obserwacja, klasa przed / po softmax
<code>y_train_onehot</code>	$(1600, 10)$	obserwacja, poprawna klasa jako jedynka

W całym modelu jest $30 \cdot 64 + 64 + 64 \cdot 10 + 10 = 2634$ uczonych liczb. Wiersz macierzy to jedna obserwacja, ale wiersz W_2 to jeden neuron ukryty; czytając $a_1 @ W_2$ nie należy mieszać tych dwóch znaczeń. *Parametry* to cztery tablice W_1, b_1, W_2, b_2 zmieniane podczas treningu. *Hiperparametry* (np. h , learning rate η , liczba epok) wybieramy przed treningiem.

3 Dane syntetyczne: klasy mają środki i szum

3.1 Najpierw prosty przykład, potem rozkład

Niech środek klasy 3 dla dwóch cech będzie $\mu_3 = (2, -1)$. Gdy wylosowany szum dla pewnego punktu to $\varepsilon = (0.3, -0.4)$, punkt ma współrzędne $x = \mu_3 + \varepsilon = (2.3, -1.4)$ i etykietę 3. Rzeczywisty kod losuje 30 liczb środka dla *każdej* klasy i 200 osobnych szumów wokół niego. Punkty jednej klasy mają więc wspólny środek, lecz różnią się odchyleniem.

```

8 def generate_synthetic_data(
9     n_classes: int = 10,
10    n_features: int = 30,
11    n_samples_per_class: int = 200,
12    center_scale: float = 1.0,
13    noise_scale: float = 2.0,
14    seed: int = 0,
15 ) -> tuple[np.ndarray, np.ndarray]:
16     np.random.seed(seed)
17
18     X_list = []
19     y_list = []
20     for class_idx in range(n_classes):
21         center = np.random.randn(n_features) * center_scale
22         points = center + np.random.randn(n_samples_per_class, n_features) * noise_scale
23         labels = np.full(n_samples_per_class, class_idx)

```

W `center` trafia jeden wektor o długości 30. `np.random.randn(n_features)` losuje liczby zwykle bliskie zeru, dodatnie i ujemne. Mnożnik `center_scale=1.0` rozsuwa lub zbliża środki. `np.random.randn(n_samples_per_class, n_features)` losuje prostokątną tablicę szumu 200×30 , a `noise_scale=2.0` określa jego typową wielkość. Dodanie środka (30,) do tablicy (200,30) stosuje ten sam wektor do każdego wiersza; to *broadcasting* NumPy. `np.full(200, class_idx)` daje etykiety pasujące do tych punktów.

Zapis probabilistyczny *po* zrozumieniu kodu jest krótki:

$$\mu_k = s_c u_k, \quad x_{ki} = \mu_k + s_n \varepsilon_{ki}, \quad u_k, \varepsilon_{ki} \sim \mathcal{N}(0, I_d), \quad s_c = 1, \quad s_n = 2. \quad (3.1)$$

Symbol $\mathcal{N}(0, I_d)$ mówi: każda składowa losowana jest z normalnego rozkładu o średniej 0 i wariancji 1; I_d zaznacza brak korelacji między składowymi tego losowania. W tym generatorze po ustaleniu wylosowanego środka rozrzut *pojedynczej* cechy wokół niego ma wariancję $s_n^2 = 4$, a nie 2. Równoważnie można pisać $x_{ki} | \mu_k \sim \mathcal{N}(\mu_k, 4I_d)$; kreska oznacza „gdy znamy środek”. To dokładnie sens formalnej notacji z wcześniejszej wersji, lecz teraz widzimy odpowiadające jej dwie linie programu.

3.2 Sklejenie, wspólne tasowanie i podział

```

25     X_list.append(points)
26     y_list.append(labels)
27
28     X = np.vstack(X_list)
29     y = np.concatenate(y_list)

```

```

30
31     perm = np.random.permutation(len(X))
32     X = X[perm]
33     y = y[perm]
34
35     return X, y
36
37
38 def main() -> None:
39     X, y = generate_synthetic_data()
40     split = int(0.8 * len(X))
41     X_train, X_val = X[:split], X[split:]
42     y_train, y_val = y[:split], y[split:]
43
44     y_train_onehot = np.zeros((len(y_train), 10))
45     y_train_onehot[np.arange(len(y_train)), y_train] = 1

```

`vstack` ustawia dziesięć tablic 200×30 jedna pod drugą, a `concatenate` skleja dziesięć wektorów etykiet; razem jest $N = 2000$ obserwacji. `perm` jest jedną losową kolejnością indeksów. To, że ta sama permutacja jest użyta dla `X` i `y`, chroni pary punkt–etykieta. Potem $0.8 \cdot 2000 = 1600$ obserwacji idzie do treningu, a 400 pozostałych służy do walidacji. Nie ma osobnego testu ani wymuszonej równej liczby klas w obu częściach. W faktycznym uruchomieniu liczności klas walidacyjnych wyniosły (45, 45, 36, 42, 37, 40, 38, 37, 43, 37).

Co zmienić, aby sprawdzić zrozumienie?

Zwiększ `noise_scale`, nie zmieniając `center_scale`. Chmury punktów bardziej na siebie najdą; można oczekiwać trudniejszej klasyfikacji, ale pojedynczy wylosowany zbiór może dać nieregularny wynik. `np.random.seed(seed)` umożliwia odtwarzanie takiego losowania kosztem resetu globalnego generatora NumPy.

4 Etykieta jako liczba i jako wektor one-hot

Numery $0, 1, \dots, 9$ to nazwy klas; klasa 9 nie jest dziewięć razy większa niż 1. Do straty sieć potrzebuje rozpisania poprawnej klasy na dziesięć pól. Dla klasy 3: $y = (0, 0, 0, 1, 0, 0, 0, 0, 0, 0)$. W kodzie:

```

y_train_onehot = np.zeros((len(y_train), 10))
y_train_onehot[np.arange(len(y_train)), y_train] = 1

```

Jeśli pierwsze trzy etykiety to $(3, 0, 2)$, zaawansowane indeksowanie wybiera kolejno komórki $(0, 3), (1, 0), (2, 2)$ i tam stawia jedynki. `y_train_onehot` ma $(1600, 10)$. Zapis $y_{ik} = 1$ oznacza, że obserwacja i należy do klasy k ; dla pozostałych kolumn $y_{ik} = 0$. Etykiety walidacyjne pozostają pojedynczymi liczbami, żeby porównać je z `predict`.

5 Neuron: ważona suma, bias i przejście do macierzy

5.1 Jedna obserwacja i jeden neuron

Powiązanie z regresją liniową jest bezpośrednie. Dla dwóch cech i wag $(0.2, 0.3)$ pierwszy neuron liczy $z = 0.2x_1 + 0.3x_2 + b$. Przy $x = (1, 1)$ oraz $b = 0$ otrzymuje $z = 0.5$. *Bias* jest przesunięciem

dodawanym nawet wtedy, gdy wszystkie cechy wynoszą zero. Dla d cech i neuronu r wzór brzmi

$$z_{1,ir} = \sum_{j=1}^d X_{ij} W_{1,jr} + b_{1,r}. \quad (5.1)$$

Symbol $\sum$ po prostu dodaje wkłady kolejnych cech; przy $d = 2$ to $x_1 W_{1,1r} + x_2 W_{1,2r}$. To samo obliczenie dla *wszystkich* neuronów i przykładów NumPy wykonuje jako $Z_1 = XW_1 + b_1$. Bias o kształcie $(64,)$ jest dodawany do każdego z B wierszy bez ręcznego kopiowania.

5.2 Po co dwie warstwy?

Gdyby między dwiema warstwami nie było funkcji nieliniowej, mielibyśmy

$$(XW_1 + b_1)W_2 + b_2 = X(W_1W_2) + (b_1W_2 + b_2), \quad (5.2)$$

czyli w praktyce jedną transformację liniową z przesunięciem. ReLU między warstwami zmienia sytuację: dla części wejść konkretny neuron zostaje wyłączony, dla innych działa. Sieć może więc wykorzystać różne zestawy aktywnych neuronów w różnych obszarach wejścia. Sama obecność warstwy ukrytej *nie* sprawia magicznie, że model lepiej klasyfikuje; daje mu dodatkową elastyczność, którą trzeba wytrenować.

6 ReLU: nieliniowość i bramka dla gradientu

```

4 def relu(x: np.ndarray) -> np.ndarray:
5     return np.maximum(x, 0)
6
7
8 def relu_backward(dout: np.ndarray, x: np.ndarray) -> np.ndarray:
9     return np.where(x >= 0, dout, 0)
```

ReLU (rectified linear unit, prostownik liniowy) to $a = \max(0, z)$. Dla $z = (-2, 0, 3)$ dostajemy $a = (0, 0, 3)$. Dodatnie wartości przechodzą bez zmiany, ujemne znikają. `np.maximum` działa komórka po komórce niezależnie od kształtu tablicy. Powstałe `a1` ma nadal kształt $(B, 64)$; podczas dalszych obliczeń każda aktywacja jest sygnałem wejściowym do drugiej warstwy.

Wstecz ReLU działa jak bramka: przy $z > 0$ niewielka zmiana z zmienia a o tyle samo, czyli pochodna wynosi 1; przy $z < 0$ wyjście jest stale 0, więc pochodna wynosi 0. Jeśli sygnał pochodnej dochodzący z *dalszej części sieci* to `dout`, rezultat jest `dout * 1[z > 0]`. Kod wybiera $x \geq 0$, a więc w *dokładnie* zerowym punkcie przepuszcza sygnał. ReLU nie ma tam klasycznej pochodnej; ten wybór jest dopuszczalnym podgradientem. Numeryczna kontrola gradientów blisko zera może dlatego różnić się od rachunku różnicą centralną. Bramka przy $z < 0$ nie znaczy, że taki neuron już nigdy się nie uaktywni: zmiany innych parametrów i innych danych mogą zmienić jego przyszłe z .

7 Początkowe wagi i architektura TwoLayerNet

```

1 import numpy as np
2 from .layers import relu, relu_backward, softmax, cross_entropy
3
4
5 class TwoLayerNet:
6     def __init__(
7         self, n_features: int, n_hidden: int, n_classes: int, seed: int = 42
```

```

8         ) -> None:
9             np.random.seed(seed)
10            self.W1 = np.random.randn(n_features, n_hidden) * 0.01
11            self.W2 = np.random.randn(n_hidden, n_classes) * 0.01
12            self.b1 = np.zeros(shape=(n_hidden,))
13            self.b2 = np.zeros(shape=(n_classes,))

```

Konstruktor otrzymuje d, h, C i `seed`. Zapis `np.random.randn(d,h) * 0.01` daje $d \cdot h$ losowych wag o małej skali, a `np.zeros` ustawia wszystkie biasy na zero. W praktycznej konfiguracji `TwoLayerNet(30,64,10)` tworzy $W_1 : (30, 64)$, $W_2 : (64, 10)$, $b_1 : (64,)$, $b_2 : (10,)$. Pierwsza warstwa ma $1920 + 64 = 1984$ parametrów; druga $640 + 10 = 650$.

Dlaczego wagi losowe, ale biasy mogą być zerowe? Gdyby wszystkie kolumny W_1 zaczynały od identycznych wartości (na przykład samych zer), wszystkie ukryte neurony widziałyby to samo i otrzymywałyby symetryczne sygnały uczenia. Losowanie łamie tę symetrię. Różne neurony mogą potem nauczyć się różnych zależności. Bias zerowy nie wymusza takiej symetrii, jeśli wagi już się różnią. Skala 0.01 sprawia, że początkowe logity są podobne, ale nie jest automatycznie najlepsza dla ReLU i każdego danych; dla głębszych sieci często rozważa się inicjalizację uwzględniającą liczbę wejść neuronu.

Wywołanie `np.random.seed(seed)` ustawia *globalny* stan losowości NumPy. Generator danych wcześniej ustawia `seed=0`, a konstruktor później `seed=42`; tasowanie w `train` korzysta ze stanu pozostawionego przez konstruktor. To tłumaczy powtarzalne wyniki w tym uruchomieniu i zarazem pokazuje, że przeniesienie lub dodanie losowania może zmienić dalszą sekwencję.

8 Przebieg w przód: od tablicy cech do wyników klas

```

15     def forward(self, X: np.ndarray) -> tuple[np.ndarray, dict]:
16         z1 = X @ self.W1 + self.b1
17         a1 = relu(z1)
18         z2 = a1 @ self.W2 + self.b2
19         probs = softmax(z2)
20         cache = {"X": X, "z1": z1, "a1": a1, "z2": z2, "probs": probs}
21
22         return probs, cache

```

Pierwsze trzy przekształcenia można przeczytać razem:

$$\begin{aligned} Z_1 &= XW_1 + b_1, & A_1 &= \text{ReLU}(Z_1), \\ Z_2 &= A_1W_2 + b_2, & P &= \text{softmax}(Z_2). \end{aligned} \quad (8.1)$$

Pierwszy iloczyn ma wymiary $(B, 30) @ (30, 64) = (B, 64)$, drugi $(B, 64) @ (64, 10) = (B, 10)$. `z1` to wartości *przed* ReLU, `a1` to po ReLU, `z2` to *logity* (surowe oceny klas), `probs` to ich udziały po softmax. `cache` zapisuje wartości pośrednie. Żadna z tych linii jeszcze nie zmienia wag: `forward` może więc służyć zarówno podczas treningu, jak i predykcji.

Mały przykład $2 \rightarrow 2 \rightarrow 2$. Weźmy $x = (1, 1)$, zerowe biasy i macierze

$$W_1 = \begin{pmatrix} 0.2 & -0.1 \\ 0.3 & 0.2 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 0.4 & -0.2 \\ -0.1 & 0.3 \end{pmatrix}. \quad (8.2)$$

Wtedy $z_1 = xW_1 = (0.5, 0.1)$ i $a_1 = (0.5, 0.1)$. Drugie mnożenie daje $z_2 = (0.5 \cdot 0.4 + 0.1 \cdot (-0.1), 0.5 \cdot (-0.2) + 0.1 \cdot 0.3) = (0.19, -0.07)$. To dokładnie te same działania co w dużej sieci, tylko z dwiema kolumnami zamiast 64 i 10. W sekcjach 9–16 przejdziemy tę samą obserwację aż do nowych wag.

9 Softmax: co oznacza wynik klasyfikatora

9.1 Od logitów do udziałów

Dla logitów $(0.19, -0.07)$ chcemy uzyskać dwie liczby dodatnie sumujące się do 1. Definiujemy

$$p_k = \frac{e^{z_k}}{\sum_{j=0}^{C-1} e^{z_j}}. \quad (9.1)$$

Mianownik to suma po *klasach tego samego punktu*. Dla naszej pary $p_0 = e^{0.19} / (e^{0.19} + e^{-0.07}) \simeq 0.564636$ i $p_1 \simeq 0.435364$. Różnica logitów wynosi 0.26; dla dwóch klas $p_0 = 1 / (1 + e^{-0.26})$. Jeśli oba logity podniesiesz o 100, *różnica* pozostanie 0.26, a wynik softmax ten sam. Najwyższy logit ma też najwyższy udział, więc *argmax* można by liczyć na obu tablicach.

```

12 def softmax(logits: np.ndarray) -> np.ndarray:
13     logits_shifted = logits - logits.max(axis=-1, keepdims=True)
14     exp_z = np.exp(logits_shifted)
15     return exp_z / exp_z.sum(axis=-1, keepdims=True)

```

W kodzie najpierw od *każdego wiersza* odejmowane jest jego maksimum. Ponieważ

$$\frac{e^{z_k - m}}{\sum_j e^{z_j - m}} = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad (9.2)$$

wynik jest matematycznie taki sam. Za to największy argument wykładnika staje się 0: ograniczamy ryzyko przepełnienia przy bardzo dużych logitach. `axis=-1` odnosi się do ostatniej osi, tutaj klas; `keepdims=True` zachowuje kolumnę $(B, 1)$ do odejmowania i dzielenia po wierszach. `exp_z` ma (B, C) ; `exp_z.sum(...)` ma $(B, 1)$, więc każda z C liczb jednego wiersza jest dzielona przez *ten sam* mianownik. Dla ekstremalnie ujemnej różnicy logitów `np.exp` może zaokrąglić wynik do zera mimo przesunięcia; wrócimy do tego przy stracie.

9.2 Udział a pewność modelu

Wartość 0.8 oznacza, że model przydzielił tej klasie 80% *we własnym rozkładzie wynikowym*. Żeby nazywać ją wiarygodnym prawdopodobieństwem częstości poprawnych decyzji, potrzebna byłaby dodatkowa ocena kalibracji na danych poza treningiem. Softmax zapewnia sumę równą 1, ale nie gwarantuje, że na 100 prognoz z udziałem 0.8 około 80 będzie poprawnych. Tak samo *argmax* mówi o wyborze klasy, nie o jakości wszystkich udziałów.

10 Entropia krzyżowa: za co model dostaje karę

10.1 Jedna etykieta, jeden logarytm

Założmy, że prawdziwa jest klasa 0. Z naszych udziałów $(0.564636, 0.435364)$ wybieramy p_0 i liczymy stratę $\ell = -\log(0.564636) \simeq 0.571573$. Gdy udział poprawnej klasy rośnie, ta liczba maleje: $-\log(0.9) \simeq 0.105$, $-\log(0.1) \simeq 2.303$. Zła prognoza stawiająca *bardzo mało* na prawdziwą klasę dostaje dużą karę. W odróżnieniu od *accuracy* strata rozróżnia trafienie „ledwie” od trafienia zdecydowanego.

Dla B obserwacji i C klas zapis z etykietą one-hot jest

$$L = -\frac{1}{B} \sum_{i=1}^B \sum_{k=0}^{C-1} y_{ik} \log p_{ik} = -\frac{1}{B} \sum_{i=1}^B \log p_{i, c_i}. \quad (10.1)$$

Wewnętrzna suma po klasach *wybiera* jeden wyraz, bo dokładnie jedno y_{ik} w wierszu ma wartość 1. Zewnętrzna suma agreguje obserwacje, a dzielenie przez B daje średnią. To jest odpowiednik MSE jako funkcji celu dla tego zadania, lecz inne są wyjścia i kary. Nie mylmy jej z walidacyjną dokładnością: strata używa udziałów, *accuracy* liczy tylko poprawne zwycięskie klasy.

```

18 def cross_entropy(y_true_onehot: np.ndarray, probs: np.ndarray) -> np.ndarray:
19     eps = 1e-9
20     per_sample = -np.sum(y_true_onehot * np.log(probs + eps), axis=1)
21     return np.mean(per_sample)

```

W kodzie $y_true_onehot * np.log(probs+eps)$ zeruje wkłady niewłaściwych kolumn, $sum(axis=1)$ zlicza po klasach w obrębie obserwacji, minus zmienia logarytm w nieujemną karę w zwykłym zakresie, a $mean$ uśrednia po punktach. $eps=1e-9$ zapobiega obliczeniu $\log 0$. Formalnie kod liczy $L_\varepsilon = -B^{-1} \sum y_{ik} \log(p_{ik} + \varepsilon)$; może ono dać niewielką wartość ujemną przy $p_{i,c_i} = 1$, bo $\log(1 + \varepsilon) > 0$. Ważniejsza konsekwencja dotyczy pochodnej, opisana w sekcji 23.

10.2 Dlaczego logarytm: jedna intuicja probabilistyczna

Jeśli model przypisze kolejnym poprawnym etykiety udziały 0.8 i 0.5, ich łączna wiarygodność przy odpowiednim uproszczeniu to $0.8 \cdot 0.5 = 0.4$. Maksymalizacja takiego iloczynu byłaby niewygodna i numerycznie niestabilna przy wielu punktach. Logarytm zmienia iloczyn w sumę: $\log(0.8 \cdot 0.5) = \log 0.8 + \log 0.5$. Minimalizacja ujemnej sumy jest równoważna maksymalizacji iloczynu, jeśli pominiemy pozytywny czynnik średniej. To *interpretacja* funkcji celu przy modelu warunkowym $p(y | x)$; generator danych nie jest tutaj estymowany jako jeden pełny model prawdopodobieństwa wraz z rozkładem cech.

Gdy każdy z $C = 10$ logitów jest prawie taki sam, $p_k \approx 1/10$ i $L \approx -\log(0.1) = \log 10 \approx 2.302585$. Dlatego `main` sprawdza początkową stratę bliską tej liczbie. Ten test nie mówi, że model już rozpoznaje klasy. Oznacza tylko sensowny stan początkowy.

11 Propagacja wsteczna: co właściwie ma policzyć?

W serii 3Blue1Brown możesz zobaczyć zmianę jednej wagi i śledzić, jak zmienia się końcowa strata. Zróbmy to na konkretnym połączeniu: $W_{2,00} = 0.4$ łączy aktywację pierwszego neuronu ukrytego $a_{1,0} = 0.5$ z logitem klasy 0. Podniesienie tej wagi o 0.01 podniesie logit $z_{2,0}$ o $0.5 \cdot 0.01 = 0.005$. Jeżeli podniesienie logitu prawdziwej klasy *obniża* stratę, zwiększenie wagi pomoże. Pochodna mówi nam, *o ile na jednostkę malej zmiany*.

Reguła łańcuchowa w wersji znanej z analizy mówi: gdy $L = L(z(w))$, to $dL/dw = (dL/dz)(dz/dw)$. W sieci istnieje kilka dróg od jednej wagi do wyniku; tam, gdzie drogi się spotykają, dodajemy ich wkłady. Gdy jedna waga jest użyta dla wielu przykładów, dodajemy także wkłady tych przykładów. To cała logika stojąca za macierzami w *backward*. Nazwa *backpropagation* oznacza oszczędne liczenie tych pochodnych od straty w kierunku wejścia, wykorzystując raz policzone wyniki pośrednie.

Pytanie o małą zmianę	Zachowana wartość z forward	Potrzebna pochodna
Jak strata zależy od ocen klas?	probs, etykieta	$\partial L / \partial Z_2$
Jak waga wyjściowa wpływa na ocenę?	a1	$\partial L / \partial W_2$
Jak oceny zależą od neuronu?	w2	$\partial L / \partial A_1$
Czy neuron przepuszcza zmianę?	z1	$\partial L / \partial Z_1$
Jak wejściowa waga wpływa na neuron?	x	$\partial L / \partial W_1$

cache z **forward** zawiera dokładnie wartości wymienione w środkowej kolumnie; **backward** nie musi ich odtwarzać. We wszystkich następnych sekcjach najpierw policzymy pojedynczą pochodną, a potem pokażemy, jak kod liczy całą macierz naraz.

12 Krok wstecz I: gradient po logitach

12.1 Dwie klasy: rachunek pochodnych bez wielkich sum

Dla jednej obserwacji z prawdziwą klasą 0 mamy

$$\ell = -\log p_0 = -\log \left(\frac{e^{z_0}}{e^{z_0} + e^{z_1}} \right) = \log(e^{z_0} + e^{z_1}) - z_0. \quad (12.1)$$

Pochodna logarytmu złożenia i pochodna e^z dają

$$\begin{aligned} \frac{\partial \ell}{\partial z_0} &= \frac{e^{z_0}}{e^{z_0} + e^{z_1}} - 1 = p_0 - 1, \\ \frac{\partial \ell}{\partial z_1} &= \frac{e^{z_1}}{e^{z_0} + e^{z_1}} = p_1. \end{aligned} \quad (12.2)$$

Dla $p = (0.564636, 0.435364)$ to $(-0.435364, +0.435364)$. Pierwszy znak mówi: *podnieś* logit poprawnej klasy (odejmując ujemny gradient). Drugi mówi: *obniż* logit złej klasy. Ich suma to 0, bo wspólne podniesienie obu logitów nie zmienia softmax. Ten sprawdzian znaków jest przydatniejszy na początek niż zapamiętanie wzoru.

12.2 Wiele klas i średnia po partii

Dla dowolnej liczby klas analogiczny rachunek daje dla obserwacji i i klasy k pochodną $p_{ik} - y_{ik}$. Dlatego kod zaczyna **backward** tak:

```

24 def backward(self, y_true_onehot: np.ndarray, cache: dict) -> dict:
25     n = y_true_onehot.shape[0]
26     d_logits = (cache["probs"] - y_true_onehot) / n

```

n to *aktualna* liczba wierszy, czyli B . Przy $B = 32$ sygnał jednej obserwacji jest 32 razy mniejszy niż bez średniej. Dla całej partii zapis macierzowy to

$$D_2 := \frac{\partial L}{\partial Z_2} = \frac{P - Y}{B}, \quad D_2 \in \mathbb{R}^{B \times C}. \quad (12.3)$$

Litera D_2 odpowiada dokładnie tablicy `d_logits`. Zapis $\partial L / \partial Z_2$ oznacza tablicę o *tych samych wymiarach* co Z_2 : w pozycji (i, k) jest wpływ małej zmiany tego logitu na jedną, uśrednioną stratę L . Dzielenie występuje już tutaj; później nie dzielimy gradientów wag drugi raz.

Jeśli chcesz zobaczyć, skąd bierze się skrót $p - y$. Dla softmax $p_k = e^{z_k}/S$, gdzie $S = \sum_j e^{z_j}$, przy zmianie z_j pochodna wynosi $\partial p_k / \partial z_j = p_k(\mathbf{1}[k = j] - p_j)$. Przykład: zwiększenie logitu klasy j podnosi jej udział, a jednocześnie zmniejsza udziały pozostałych przez wspólny mianownik. Podstawienie tego do pochodnej $-\sum_k y_k \log p_k$ daje

$$\frac{\partial \ell}{\partial z_j} = -\sum_k y_k(\mathbf{1}[k = j] - p_j) = -y_j + p_j \sum_k y_k = p_j - y_j. \quad (12.4)$$

Ostatni krok korzysta z tego, że etykieta one-hot sumuje się do 1. Nie musisz zapamiętywać pochodnych każdej składowej softmax; wystarczy odtworzyć rachunek dwóch klas i rozumieć rolę mianownika.

13 Krok wstecz II: wagi i biasy drugiej warstwy

W drugiej warstwie każda ocena klasy k jest sumą wkładów neuronów:

$$z_{2,ik} = \sum_{r=1}^h a_{1,ir} W_{2,rk} + b_{2,k}. \quad (13.1)$$

Dla jednej wagi $W_{2,rk}$ zmiana o małą wartość Δw zmienia logit tej klasy o $a_{1,ir} \Delta w$. Pochodna logitu względem wagi to więc $a_{1,ir}$, a reguła łańcuchowa daje wkład obserwacji i równy $a_{1,ir} D_{2,ik}$. Ponieważ jedna waga jest użyta dla *każdego* wiersza, sumujemy:

$$\frac{\partial L}{\partial W_{2,rk}} = \sum_{i=1}^B a_{1,ir} D_{2,ik}, \quad \frac{\partial L}{\partial b_{2,k}} = \sum_{i=1}^B D_{2,ik}. \quad (13.2)$$

Pierwsze równanie znaczy *aktywacja razy sygnał klasy, dodane po przykładach*. Bias wchodzi z współczynnikiem 1 zamiast aktywacji, dlatego drugi wzór jest samą sumą sygnałów. Na przykład przy $B = 1$, $a_{1,0} = 0.5$ i $D_{2,0} = -0.435364$, gradient $W_{2,00}$ wynosi $0.5 \cdot (-0.435364) = -0.217682$.

```
27 dW2 = cache["a1"].T @ d_logits
28 db2 = d_logits.sum(axis=0)
```

Macierzowy odpowiednik (13.2) to $\nabla_{W_2} L = A_1^\top D_2$. Po transpozycji $A_1^\top$ ma kształt (h, B) i mnoży (B, C) ; wymiar B w iloczynie to dokładnie suma po obserwacjach. Wynik (h, C) pasuje do W_2 . W `sum(axis=0)` zerowa oś oznacza wiersze obserwacji; rezultat $(C,)$ pasuje do `b2`. Tak samo działają te linie dla $B = 1$, 32 czy 16 w kontroli gradientu.

13.1 Miniaturowa partia: po co jest suma?

Załóżmy, że dla tej samej wagi drugi punkt ma aktywację $a_{1,0}^{(2)} = 0.2$ i sygnał $D_{2,0}^{(2)} = +0.05$, podczas gdy pierwszy ma $a_{1,0}^{(1)} = 0.5$ i $D_{2,0}^{(1)} = -0.10$. Wspólny gradient to $0.5(-0.10) + 0.2(0.05) = -0.04$: wpływy punktów mogą się częściowo znosić. Liczby D_2 w tym przykładzie *zawierają już dzielenie przez B* ; nie dodawaj kolejnego $1/2$. Właśnie to robi `a1.T @ d_logits`; nie jest to tylko skrót składniowy, lecz zapis sumy wkładów korzystających z tej samej wagi.

14 Krok wstecz III: sygnał dla neuronów ukrytych

Zmiana jednej aktywacji $a_{1,ir}$ wpływa na *wszystkie* C logitów przez wiersz $W_{2,r\cdot}$. Dlatego, trzymając się reguły łańcuchowej, dodajemy wpływy klas:

$$\frac{\partial L}{\partial a_{1,ir}} = \sum_{k=0}^{C-1} D_{2,ik} W_{2,rk}. \quad (14.1)$$

Dla pierwszego neuronu w naszym przykładzie dwa wkłady to $(-0.435364) \cdot 0.4$ i $(+0.435364) \cdot (-0.2)$. Suma wynosi -0.261218 . Gdyby wziąć tylko drogę przez poprawną klasę, gradient byłby błędny: aktywacja wpływa także na ocenę konkurencyjnej klasy.

```
d_a1 = d_logits @ self.W2.T
```

Wymiary $(B, C) @ (C, h) = (B, h)$ odpowiadają tablicy aktywacji. Transpozycja `W2.T` jest potrzebna, bo poruszamy się *od wyników klas do neuronów*. To samo równanie działa przy dowolnej liczbie klas i neuronów. `d_a1` jest pochodną po *wyjściu ReLU*; zanim przejdziemy do wcześniejszych wag, musimy uwzględnić bramkę z sekcji 6.

15 Krok wstecz IV: ReLU i pierwsza warstwa

```
30 d_z1 = relu_backward(d_a1, cache["z1"])
31 dW1 = cache["X"].T @ d_z1
32 db1 = d_z1.sum(axis=0)
33 return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}
```

`relu_backward(d_a1, cache["z1"])` mnoży otrzymany sygnał przez maskę aktywności. Matematycznie

$$D_{1,ir} = D_{A_1,ir} \mathbf{1}[z_{1,ir} \geq 0]. \quad (15.1)$$

W przykładzie $z_1 = (0.5, 0.1)$, więc oba neurony były aktywne i $D_1 = (-0.261218, +0.174145)$. Gdyby drugi z_1 był ujemny, jego sygnał do W_1 stałby się zerem dla *tej obserwacji*. Drugi etap jest analogiczny do wyprowadzenia dla W_2 : z równania (5.1) zmiana $W_{1,jr}$ o Δw przesuwą $z_{1,ir}$ o $X_{ij}\Delta w$. Stąd

$$\frac{\partial L}{\partial W_{1,jr}} = \sum_{i=1}^B X_{ij} D_{1,ir}, \quad \frac{\partial L}{\partial b_{1,r}} = \sum_{i=1}^B D_{1,ir}. \quad (15.2)$$

Na przykład dla $X_{00} = 1$ gradient $W_{1,00}$ wynosi $1(-0.261218) = -0.261218$. Macierzowy kod `X.T @ d_z1` mnoży $(d, B) @ (B, h)$ i oddaje (d, h) , tak jak W_1 ; `sum(axis=0)` daje $(h,)$, tak jak b_1 . Metoda `backward` zwraca cztery gradienty; żadnego parametru jeszcze nie zmienia.

Co mówił skomplikowany wzór z indeksami? Cały zapis $\sum_i X_{ij} D_{1,ir}$ to po prostu „dla wejścia j i neuronu r : pomnóż wartość wejścia przez sygnał tego neuronu i dodaj wyniki po punktach”. Symbole i, j, r służą jedynie pilnowaniu, *po czym sumujemy*, a *co zostaje* w macierzy. `X.T @ d_z1` liczy naraz wszystkie kombinacje (j, r) . Matematyka i implementacja opisują ten sam rachunek, a nie dwa oddzielne algorytmy.

15.1 Podsumowanie pochodnych i wymiarów

Kod	Wzór	Kształt
<code>d_logits</code>	$(P - Y)/B$	(B, C)
<code>dW2</code>	$A_1^\top D_2$	(h, C)
<code>db2</code>	$\sum_i D_{2,i}$	$(C,)$
<code>d_a1</code>	$D_2 W_2^\top$	(B, h)
<code>d_z1</code>	$D_{A_1} \odot \mathbf{1}[Z_1 \geq 0]$	(B, h)
<code>dW1</code>	$X^\top D_1$	(d, h)
<code>db1</code>	$\sum_i D_{1,i}$	$(h,)$

Symbol $\odot$ oznacza mnożenie komórka po komórce. Sprawdzenie kształtu każdej tablicy jest praktycznym testem błędów: gradient wagi musi mieć ten sam kształt co waga. Przy pierwszym

czytaniu możesz zatrzymać się na formie pojedynczej obserwacji; macierz jest szybkim sposobem powtórzenia tego rachunku dla całej partii.

16 Jeden pełny krok uczenia, policzony na liczbach

Wracamy do (8.2): $B = 1$, $x = (1, 1)$, poprawna klasa 0, $y = (1, 0)$, biasy zerowe. Przebieg w przód dał $z_1 = (0.5, 0.1)$, $a_1 = (0.5, 0.1)$, $z_2 = (0.19, -0.07)$, $p = (0.56463629, 0.43536371)$ i stratę kodu $-\log(p_0 + 10^{-9}) = 0.57157348$. Wszystkie poniższe gradienty to dokładnie wartości zwracane przez `backward` z zaokrągleniem.

$$D_2 = p - y = (-0.43536371, 0.43536371), \quad (16.1)$$

$$\nabla W_2 = a_1^\top D_2 = \begin{pmatrix} -0.21768185 & 0.21768185 \\ -0.04353637 & 0.04353637 \end{pmatrix}, \quad \nabla b_2 = (-0.43536371, 0.43536371), \quad (16.2)$$

$$D_{A_1} = D_2 W_2^\top = (-0.26121822, 0.17414548), \quad D_1 = D_{A_1}, \quad (16.3)$$

$$\nabla W_1 = x^\top D_1 = \begin{pmatrix} -0.26121822 & 0.17414548 \\ -0.26121822 & 0.17414548 \end{pmatrix}, \quad \nabla b_1 = (-0.26121822, 0.17414548). \quad (16.4)$$

Przyjmijmy *tylko w przykładzie* $\eta = 0.1$. Reguła aktualizacji to *nowy parametr = stary parametr* $-\eta \cdot$ *gradient*. Na przykład $W_{2,00}$ rośnie z 0.4 do $0.4 - 0.1(-0.21768185) = 0.42176819$, podczas gdy $W_{1,01}$ zmienia się z -0.1 do $-0.1 - 0.1(0.17414548) = -0.11741455$. Wszystkie zmiany razem:

$$W_1^{\text{new}} = \begin{pmatrix} 0.22612182 & -0.11741455 \\ 0.32612182 & 0.18258545 \end{pmatrix}, \quad b_1^{\text{new}} = (0.02612182, -0.01741455), \quad (16.5)$$

$$W_2^{\text{new}} = \begin{pmatrix} 0.42176819 & -0.22176819 \\ -0.09564636 & 0.29564636 \end{pmatrix}, \quad b_2^{\text{new}} = (0.04353637, -0.04353637). \quad (16.6)$$

Po *nowym forward* otrzymujemy

$$z_1^{\text{new}} = (0.57836547, 0.04775636), \quad z_2^{\text{new}} = (0.28290480, -0.15768044), \quad (16.7)$$

$$p^{\text{new}} = (0.60839847, 0.39160153), \quad L^{\text{new}} = 0.49692523. \quad (16.8)$$

Udział poprawnej klasy wzrósł z 0.5646 do 0.6084, strata spadła. Wynik został policzony tą samą implementacją i sprawdzony liczbowo. Nie obiecuje on, że każdy krok na dowolnej partii obniży stratę *wszystkich* danych; w projekcie trening używa ponadto $\eta = 0.01$, a nie przykładowego 0.1.

17 Dwa punkty naraz: skąd biorą się sumy i transpozycje

Poprzednio liczyliśmy gradient dla *jednej* obserwacji. Teraz sprawdzmy drugi punkt, aby zobaczyć, co naprawdę znaczą tablice NumPy. Zachowujemy wagi i biasy z (8.2), ale bierzemy

$$X = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad Y = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}. \quad (17.1)$$

Pierwszy punkt ma etykietę 0 jak wcześniej; drugi ma etykietę 1. Mnożenie $x @ w_1$ oblicza oba zestawy neuronów jednocześnie. Wyniki są

$$Z_1 = \begin{pmatrix} 0.5 & 0.1 \\ -0.1 & -0.3 \end{pmatrix}, \quad A_1 = \begin{pmatrix} 0.5 & 0.1 \\ 0 & 0 \end{pmatrix}, \quad (17.2)$$

$$Z_2 = \begin{pmatrix} 0.19 & -0.07 \\ 0 & 0 \end{pmatrix}, \quad P \simeq \begin{pmatrix} 0.564636 & 0.435364 \\ 0.5 & 0.5 \end{pmatrix}. \quad (17.3)$$

Drugi wiersz A_1 jest zerowy, bo oba jego neurony mają ujemne wartości przed ReLU. Biasy wyjściowe są zerowe, zatem jego logity i udziały są równe. Średnia strata tej *dwuelementowej* partii to $[0.571573 + 0.693147]/2 \simeq 0.632360$. Nie ma tutaj żadnej ukrytej pętli po punktach w kodzie forward: macierzowe działania liczą oba wiersze razem, a softmax działa osobno w każdym.

17.1 Sygnały mogą walczyć ze sobą

W $d_logits=(P-Y)/2$ wiersze wynoszą

$$D_2 \simeq \begin{pmatrix} -0.217682 & 0.217682 \\ 0.25 & -0.25 \end{pmatrix}. \quad (17.4)$$

Pierwszy punkt chce podnieść ocenę klasy 0, drugi chce podnieść ocenę klasy 1. Gradient biasu wyjściowego jest sumą wierszy: $\nabla b_2 \simeq (0.032318, -0.032318)$. Względem klasy 0 wkład drugiej obserwacji $+0.25$ nieco przeważa nad pierwszą -0.217682 . To tłumaczy, dlaczego gradient partii nie musi poprawiać wyniku *każdego* jej punktu z osobna.

Gradient W_2 ma natomiast tylko wkład pierwszego punktu:

$$\nabla W_2 = A_1^\top D_2 \simeq \begin{pmatrix} -0.108841 & 0.108841 \\ -0.021768 & 0.021768 \end{pmatrix}. \quad (17.5)$$

Dlaczego drugi punkt nie zmienia bezpośrednio tych wag? Jego aktywacje to $(0,0)$, więc w równaniu (13.2) każdy iloczyn $a_{1,2r}D_{2,2k}$ jest zerowy. Mimo to *bias* b_2 otrzymuje jego wkład, bo do logitów jest dodawany niezależnie od aktywacji. Na tym etapie widać sens oddzielnych gradientów wag i biasów.

17.2 Bramka ReLU w partii i sens $x.T$

Dla drugiego punktu sygnał dochodzący do aktywacji ukrytych *przed bramką* może być niezerowy, lecz $z1=(-0.1,-0.3)$ zamyka oba neurony. Jego wiersz D_1 jest więc zerowy. Pierwszy punkt wnosi swoje znane sygnały podzielone przez 2, skąd

$$D_1 \simeq \begin{pmatrix} -0.130609 & 0.087073 \\ 0 & 0 \end{pmatrix}, \quad \nabla W_1 = X^\top D_1 \simeq \begin{pmatrix} -0.130609 & 0.087073 \\ -0.130609 & 0.087073 \end{pmatrix}. \quad (17.6)$$

W $x.T$ wiersze to *cechy*, a kolumny to *obserwacje*. Po pomnożeniu przez D_1 wymiar obserwacji znika przez sumowanie, zostawiając macierz *cecha* $\times$ *neuron*. Możesz ręcznie policzyć np. komórkę $(0,0)$: $1(-0.130609) + 1(0) = -0.130609$. Komórka $(1,0)$: $1(-0.130609) + (-1)(0) = -0.130609$. Inny znak drugiej cechy nie zmienia wyniku, bo drugi punkt ma zamkniętą bramkę. Ta konkretna dwuwierszowa tablica jest praktycznym odczytaniem wzoru z sumą (15.2).

W większej partii jedne neurony są włączone dla jednych punktów, wyłączone dla innych; maska ReLU ma kształt (B,h) i każda jej komórka jest podejmowana osobno. To wyjaśnia, dlaczego nie da się zastąpić `relu_backward` jedną decyzją „ten neuron zawsze działa” dla całego treningu.

18 Uczenie na partiach: epoka, krok i aktualizacja

18.1 Dlaczego gradient liczymy na 32 punktach?

Pełen gradient dla 1600 obserwacji potrzebowałby przejścia przez cały zbiór przed każdą zmianą wag. Pojedynczy punkt dawałby tani, lecz bardzo zmienny kierunek. *Mini-batch* (mała partia)

o rozmiarze 32 jest kompromisem: na jednym kroku liczymy średnią po 32 punktach, aktualizujemy wagi i bierzemy następną partię. Dla zbioru 1600 wierszy jest $1600/32 = 50$ aktualizacji na epokę oraz 1500 na 30 epok. *Epoka* to jedno przejście przez zbiór, a *krok* to jedna zmiana parametrów. Rozróżnienie jest ważne przy czytaniu wykresu: jeden punkt wykresu odpowiada epoce, w której zaszło 50 zmian.

Formalnie dla parametrów θ i partii S_t z B indeksami:

$$g_t = \frac{1}{B} \sum_{i \in S_t} \nabla_{\theta} \ell_i(\theta_t), \quad \theta_{t+1} = \theta_t - \eta g_t. \quad (18.1)$$

W backward dzielenie przez B jest już częścią g_t . Symbol η to *learning rate*, wielkość pojedynczego kroku. Duża wartość może przechodzić za daleko, mała może uczyć bardzo wolno. `train` ma domyślnie `lr=0.5`, ale `main` przekazuje `lr=0.01` i ta druga wartość obowiązuje w omawianym uruchomieniu. Nie ma tutaj Momentum, Adama ani adaptacji η .

18.2 Tasowanie i wycinanie partii

```

6 def train(
7     model: "TwoLayerNet",
8     X_train: np.ndarray,
9     y_train_onehot: np.ndarray,
10    X_val: np.ndarray,
11    y_val: np.ndarray,
12    lr: float = 0.5,
13    batch_size: int = 32,
14    n_epochs: int = 30,
15 ) -> dict:
16     train_loss = []
17     val_accuracy = []
18
19     for epoch in range(n_epochs):
20         batch_loss = []
21         indices = np.random.permutation(X_train.shape[0])
22         X_shuffled = X_train[indices]
23         y_shuffled = y_train_onehot[indices]
24         for batch in range(0, X_train.shape[0], batch_size):
25             X_batch = X_shuffled[batch : batch + batch_size]
26             y_batch = y_shuffled[batch : batch + batch_size]
```

Listy `train_loss` i `val_accuracy` zaczną się puste. W każdej epoce `np.random.permutation` wybiera nową kolejność indeksów. `X_train[indices]` oraz `y_train_onehot[indices]` stosują tę samą kolejność: losowanie nie miesza etykiet *niezależnie* od punktów. Wycinki `[batch:batch+batch_size]` dają kolejne partie: przy 1600 wierszach początki to 0, 32, 64, ..., 1568. Dla $N = 70$ i wielkości 32 ostatni wycinek `[64:96]` po prostu zawierałby 6 wierszy; backward używa ich faktycznej liczby. `epoch` jest licznikiem pętli, ale nie pojawia się wewnątrz wzoru uczenia.

18.3 Gdzie naprawdę zmieniają się parametry?

```

27     probs, cache = model.forward(X_batch)
28     grads = model.backward(y_batch, cache)
29     params = model.params()
30     for name in params:
31         params[name] -= lr * grads[name]
32
33     batch_loss.append(cross_entropy(y_batch, probs))
34     train_loss.append(np.mean(batch_loss))
```

```

35     preds = model.predict(X_val)
36     val_accuracy.append((preds == y_val).mean())
37
38
39     return {"train_loss": train_loss, "val_accuracy": val_accuracy}

```

`model.forward` zwraca udziały i pamięć pośrednią, `model.backward` gradienty. Metoda `params()` zwraca słownik zawierający *referencje do tych samych tablic* `self.W1`, `self.b1`, `self.W2`, `self.b2`. To właśnie `params[name] -= lr * grads[name]` odpowiada równaniu (18.1) i zmienia je *w miejscu*. Gdyby zamiast operacji `-=` przypisać nową tablicę tylko do `params[name]`, słownik lokalny wskazywałby nowe dane, lecz pola modelu mogłyby pozostać bez zmiany. Nazwy gradientów zwracane przez `backward` muszą pasować do kluczy słownika parametrów. `train` zmienia przekazany obiekt modelu, mimo że sama funkcja zwraca tylko historię.

Subtelność zapisu straty. Po aktualizacji kod wywołuje `cross_entropy(y_batch, probs)`, ale `probs` zostało policzone *przed* tą aktualizacją i nie zmienia się magicznie po zmianie wag. W epoce kolejne partie widzą już różne parametry. `train_loss.append(np.mean(batch_loss))` to średnia 50 strat *z różnych chwil*, nie strata całego treningu policzona raz po epoce. Tu wszystkie partie mają 32 wiersze, więc zwykła średnia partii waży obserwacje równo; przy niepełnej ostatniej partii nie byłoby już tak dokładnie. Do pomiaru końcowej straty epoki należałoby jeszcze raz uruchomić `forward(X_train)` po wykonaniu wszystkich kroków.

Na koniec epoki `model.predict(X_val)` używa już *końcowego* stanu modelu. Wyrażenie `(preds == y_val).mean()` zamienia tablicę `True/False` na ułamek poprawnych decyzji: $333/400 = 0.8325$ w końcu eksperymentu. Dokładność liczona na walidacji nie wchodzi do `backward`; służy obserwacji zachowania poza zestawem wykorzystywanym do zmian wag.

19 Predykcja i dostęp do parametrów

```

35     def predict(self, X: np.ndarray) -> np.ndarray:
36         probs = self.forward(X)[0]
37         return probs.argmax(axis=1)
38
39     def params(self) -> dict:
40         return {"W1": self.W1, "b1": self.b1, "W2": self.W2, "b2": self.b2}

```

`predict` wywołuje `forward` i bierze `[0]`, czyli tablicę `probs`, z pary `(probs, cache)`. `argmax(axis=1)` wybiera największy udział *w każdym wierszu*; przy $B = 3$ daje trzy numery klas. Jeśli równe maksimum zajmują dwie kolumny, NumPy zwraca pierwszą. Cache jest obliczane również podczas predykcji, choć metoda go wtedy nie potrzebuje. `params` tworzy nowy słownik, lecz nie kopiuje zawartych w nim tablic. Dlatego modyfikacja `params()["W1"][0,0]` modyfikowałaby rzeczywistą wagę modelu. To własność implementacji, którą warto sprawdzić małym testem, żeby zrozumieć aktualizację w poprzedniej sekcji.

20 Sprawdzenie gradientu: niezależny pomiar pochodnej

20.1 Jedna waga, dwa dodatkowe przebiegi

Jeśli ręcznie wyprowadziliśmy pochodną $\partial L / \partial w$, możemy ją niezależnie *oszacować* bez back-propagation. Dla małej liczby δ najpierw lekko zwiększamy wybraną wagę, mierzymy stratę, potem lekko zmniejszamy ją względem wartości pierwotnej i mierzymy znowu:

$$g_{\text{num}}(w) = \frac{L(w + \delta) - L(w - \delta)}{2\delta}, \quad \delta = 10^{-5}. \quad (20.1)$$

Na przykład dla $L(w) = w^2$, $w = 3$, $\delta = 0.01$, dostajemy $[(3.01)^2 - (2.99)^2]/0.02 = 6$, zgodnie z pochodną $2w = 6$. To *różnica centralna*; nie uczy sieci, lecz kontroluje, czy kod obliczający gradient ma właściwy znak, skalę i zależności.

```

42     def gradient_check(
43         self, X: np.ndarray, y_true_onehot: np.ndarray, eps: float = 1e-5
44     ) -> dict:
45         cache = self.forward(X)[1]
46         analytic_grads = self.backward(y_true_onehot, cache)
47
48         relative_errors = {}
49
50         for key in analytic_grads.keys():
51             param = getattr(self, key)
52             numeric = np.zeros_like(param)
53
54             it = np.nditer(param, flags=["multi_index"])
55             for _ in it:
56                 idx = it.multi_index
57                 original = param[idx]
58                 param[idx] = original + eps
59                 loss_plus = cross_entropy(y_true_onehot, self.forward(X)[0])
60                 param[idx] = original - eps
61                 loss_minus = cross_entropy(y_true_onehot, self.forward(X)[0])
62                 param[idx] = original
63                 numeric[idx] = (loss_plus - loss_minus) / (2 * eps)
64
65             analytic = analytic_grads[key]
66             relative_errors[key] = np.linalg.norm(numeric - analytic) / (
67                 np.linalg.norm(numeric) + np.linalg.norm(analytic)
68             )
69
70     return relative_errors

```

Funkcja najpierw zapamiętuje `analytic_grads` z `backward`. Następnie iterator `np.nditer` z flagą `multi_index` odwiedza każdą pozycję czterech tablic parametrów. `original` chroni pierwotną wartość; przypisania z $+\delta$ i $-\delta$ powodują dwa nowe `forward` i pomiary straty. Przed przejściem do kolejnej pozycji waga jest przywracana. Dla 2634 wag i biasów oznacza to 5268 dodatkowych przebiegów po *tych samych* 16 obserwacjach. Kontrola jest celowo kosztowna i nie powinna działać przy każdym kroku treningu.

20.2 Błąd względny i jego ograniczenia

Wynik dla każdej tablicy to

$$r = \frac{\|G_{\text{num}} - G_{\text{ana}}\|}{\|G_{\text{num}}\| + \|G_{\text{ana}}\|}. \quad (20.2)$$

`np.linalg.norm` dla macierzy używa normy Frobeniusa (pierwiastek z sumy kwadratów elementów), a dla wektora zwykłej normy euklidesowej. Mała wartość r oznacza, że *całe* tablice gradientów są bliskie w tej normie. Dla sprawdzonej rewizji otrzymano kolejno dla (W_1, b_1, W_2, b_2) wartości około $(5.27, 5.63, 5.09, 5.03) \cdot 10^{-9}$, dużo poniżej progu 10^{-4} z asercji.

To jest mocna kontrola dla początkowego modelu i 16 punktów, a nie matematyczny dowód poprawności wszystkich możliwych wejść. Dla zbyt małego δ błędy zaokrągleń psują różnicę ilorazową, a przy dokładnym zerze ReLU funkcja nie ma zwykłej pochodnej. Jeśli obie porównywane normy są zerowe, mianownik (20.2) wynosi zero i kod nie ma osłony. Wyjątek podczas pomiaru przed przywróceniem `original` również mógłby pozostawić parametr zmieniony. Oba przypadki są ograniczeniami testu, nie opisem zwykłego uruchomienia.

20.3 Kontrola jednej rzeczywistej wagi na liczbach

Weźmy ponownie sieć $2 \rightarrow 2 \rightarrow 2$ i punkt z sekcji 16. Analityczny gradient dla $W_{2,00}$ wynosi -0.2176818541 . Możemy użyć tej samej funkcji `cross_entropy`, przestawić *wyłącznie* tę wagę z 0.4 najpierw na $0.4 + \delta$, potem na $0.4 - \delta$, i podstawiać dwie otrzymane straty do (20.1). Zmierzone pochodne:

δ	Różnica centralna	Gradient backward
10^{-2}	-0.2176819199	-0.2176818541
10^{-3}	-0.2176818544	-0.2176818541
10^{-5}	-0.2176818537	-0.2176818541

Przy $\delta = 0.01$ różnica jest widoczna dopiero po wielu cyfrach; dla dwóch mniejszych przesunięć zgoda jest jeszcze bliższa, choć nie można w nieskończoność zmniejszać δ w arytmetyce komputera. W przykładzie udział poprawnej klasy jest daleki od 10^{-9} , więc nieścisłość z (23.1) jest o wiele mniejsza od błędów widocznych w tej tabeli. To naoczna kontrola znaczenia liczby zwracanej przez `backward`, a nie dodatkowa metoda treningu.

21 Jak main składa eksperyment w całość

```

38 def main() -> None:
39     X, y = generate_synthetic_data()
40     split = int(0.8 * len(X))
41     X_train, X_val = X[:split], X[split:]
42     y_train, y_val = y[:split], y[split:]
43
44     y_train_onehot = np.zeros((len(y_train), 10))
45     y_train_onehot[np.arange(len(y_train)), y_train] = 1
46
47     model = TwoLayerNet(30, 64, 10)
48     probs, cache = model.forward(X_train)
49     loss = cross_entropy(y_train_onehot, probs)
50     assert np.isclose(loss, np.log(10), atol=0.05)
51
52     grad_errors = model.gradient_check(X_train[:16], y_train_onehot[:16])
53     assert all(err < 1e-4 for err in grad_errors.values())
54
55     history = train(model, X_train, y_train_onehot, X_val, y_val, lr=0.01, n_epochs=30)
56     assert history["val_accuracy"][-1] > 0.6
57     plot_training_curves(history["train_loss"], history["val_accuracy"])
58
59
60 if __name__ == "__main__":
61     main()

```

Pierwszy blok tworzy dane i one-hot opisane wcześniej. Następny wybiera architekturę $30 \rightarrow 64 \rightarrow 10$, oblicza początkową stratę i sprawdza, czy jest bliska $\log 10$. Przy niemal równych logitach udział każdej klasy wynosi około 0.1 ; `np.isclose(..., atol=0.05)` pozwala na odchylenie wynikające z losowania wag i danych. `cache` z tego początkowego `forward` jest obliczone, ale potem nieużywane. To asercja *rozsądnej inicjalizacji*, nie pełny test poprawności.

Kolejny blok wywołuje `gradient_check` na pierwszych 16 punktach zbioru po tasowaniu. Po pozytywnym sprawdzeniu rusza `train(..., lr=0.01, n_epochs=30)`. Warunek `val_accuracy[-1] > 0.6` oznacza oczekiwane minimum wyniku w *tych* syntetycznym eksperymencie, a nie uniwersalny próg dla sieci neuronowych. Na końcu zapisuje wykres. Konstrukcja `if __name__ == "__main__":` pozwala uruchomić program jako moduł, ale samo zaimportowanie funkcji z pliku nie rozpocznie

trenowania. Asercje Pythona mogą zostać wyłączone opcją `-O`; pełnią tu rolę szybkich kontroli w eksperymencie.

22 Wizualizacja i interpretacja rzeczywistego przebiegu

22.1 Co zapisuje `visualize.py`?

```

1 from pathlib import Path
2 import matplotlib.pyplot as plt
3
4 FIGURES_DIR = Path(__file__).parent / "figures"
5
6
7 def plot_training_curves(train_loss: list[float], val_accuracy: list[float]) -> None:
8     FIGURES_DIR.mkdir(exist_ok=True)
9     fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, sharex=True)
10    ax1.plot(train_loss)
11    ax1.set_title("Train loss")
12    ax1.set_ylabel("loss (cross-entropy)")
13
14    ax2.plot(val_accuracy)
15    ax2.set_title("Val accuracy")
16    ax2.set_xlabel("epoch")
17    ax2.set_ylabel("accuracy")
18    fig.savefig(FIGURES_DIR / "training_curves.png")

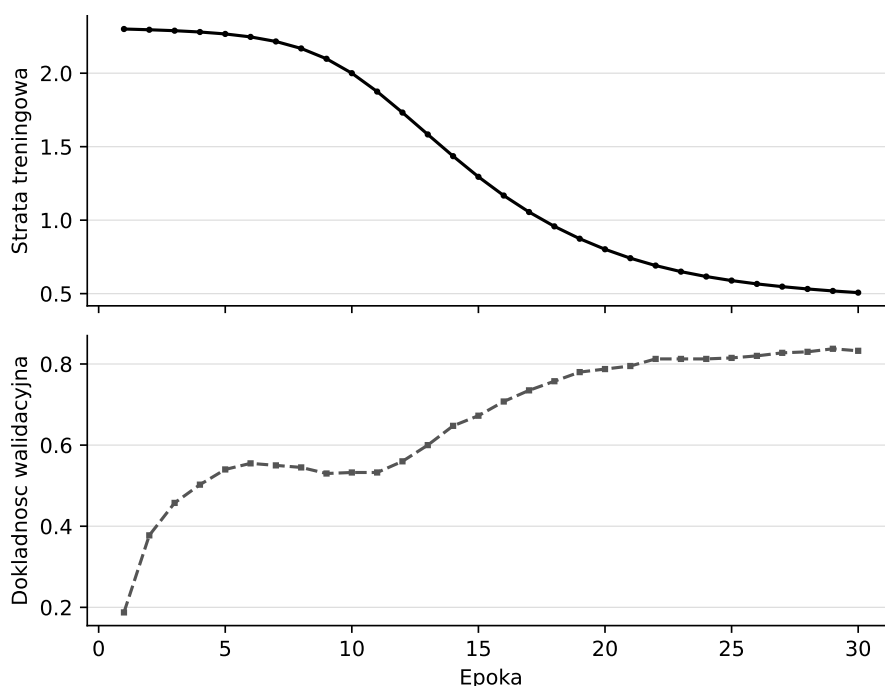
```

`Path(__file__).parent / "figures"` lokuje wynik obok modułu, a `mkdir(exist_ok=True)` zakłada katalog, jeśli go nie było. `plt.subplots(nrows=2,ncols=1,sharex=True)` buduje górny wykres straty i dolny dokładności o wspólnej osi. `ax1.plot(train_loss)` oraz `ax2.plot(val_accuracy)` dostają po 30 liczb. Oryginalny kod rysuje ich indeksy 0, ..., 29, chociaż interpretujemy je jako epoki 1, ..., 30. `fig.savefig` zapisuje PNG, przy kolejnym uruchomieniu pod tą samą nazwą go nadpisuje; figura nie jest jawnie zamykana. `__init__.py` jest pusty i nie zawiera logiki obliczeń.

22.2 Wyniki z analizowanej rewizji

Uruchomienie modułu na kodzie `32c0899c48ad` dało początkową stratę 2.302763, zgodną z intuicją log 10. Przed treningiem dokładność walidacyjna wynosiła 0.0975. Po epokach:

Epoka	Średnia strat partii w epoce	Dokładność walidacji	Trafne z 400
1	2.300698	0.1875	75
10	2.000069	0.5325	213
20	0.801843	0.7875	315
30	0.506695	0.8325	333



Rysunek 1: Krzywe odtworzone z list zwróconych przez `train`. Tutaj epoki opisano 1–30; oryginalny wykres z `visualize.py` używa indeksów 0–29.

Górna linia pokazuje coraz mniejszą średnią kar za udział prawdziwej klasy. Dolna pokazuje rosnący odsetek wybranych poprawnych etykiet. Trafienie jednego dodatkowego punktu zmienia wynik walidacyjny o $1/400 = 0.0025$. Wartości nie muszą zmieniać się monotonicznie: *accuracy* zależy od przekroczenia granicy `argmax`, strata od całego rozkładu udziałów. Na przykład model może podnieść udział prawdziwej klasy z 0.02 do 0.3 i zmniejszyć stratę, lecz nadal wybrać złą klasę z udziałem 0.4. Możliwy jest też odwrotny ruch dla niewielkiej liczby przykładów.

Czego z wykresu nie wynika? Przedtreningową stratę zmierzono raz na całym zbiorze, a `train_loss` agreguje partie w kolejnych stanach wag, zatem tych pomiarów nie można porównywać jako identycznie zdefiniowanych punktów czasowych. Walidację policzono na *jednym* wylosowanym podziale; nie ma niezależnego zbioru testowego, powtórzeń na różnych losowaniach ani przedziału niepewności dla ogólnego wyniku. Obserwacja 83.25% świadczy, że ta sieć nauczyła się rozróżniać klasy *w tym przebiegu*, nie że uzyska taką jakość na dowolnych rzeczywistych danych.

23 Dlaczego mała różnica w kodzie straty ma znaczenie?

Warto odróżniać *funkcję, którą program mierzy*, od *wzoru, którym program kieruje zmianą wag*. Sekcja 10 pokazuje, że kodowana strata to $-\log(p_c + \varepsilon)$ dla poprawnej klasy c , natomiast `backward` używa $p - y$, które wyprowadziliśmy dla $-\log p_c$. Przy normalnych wartościach p_c i $\varepsilon = 10^{-9}$ rozbieżność jest mikroskopijna. To dlatego sprawdzenie gradientów z sekcji 20 przechodzi.

Możemy dokładnie ustalić, *kiedy* wzory się rozchodzą. Pochodna idealnej straty po logitach to $p_j - y_j$. Przy dodanym ε i jednej poprawnej klasie c , reguła łańcuchowa dla $-\log(p_c + \varepsilon)$ daje

$$\frac{\partial \ell_\varepsilon}{\partial z_j} = -\frac{1}{p_c + \varepsilon} \frac{\partial p_c}{\partial z_j} = \frac{p_c}{p_c + \varepsilon} (p_j - y_j). \quad (23.1)$$

Mnożnik $p_c/(p_c + \varepsilon)$ jest prawie 1, jeżeli $p_c = 0.1$; jest bliski $1/2$, jeżeli $p_c = \varepsilon$. Przy ekstremalnie małym udziale poprawnej klasy `backward` nie jest więc dokładną pochodną funkcji `cross_entropy`. Nie oznacza to, że przykład z sekcji 16 był błędny: używa on *faktycznych* gradientów programu, a dla $p_c \approx 0.565$ różnica ginie przy podanym zaokrągleniu.

Używana w praktyce stabilna postać straty dla *logitów* mogłaby liczyć $-z_c + \log \sum_k e^{z_k}$ bez zapisywania skrajnie małego p_c ; obliczeniowo logarytm sumy wykładników też wymaga przesunięcia o maksimum. Tutaj omawiamy ją jako *możliwe ulepszenie*, nie jako kod już obecny w repozytorium. `softmax` z odjęciem maksimum zapobiega overflow wykładników, lecz dla bardzo złej klasy część prawdopodobieństw może nadal numerycznie spaść do zera. `eps` zapewnia wtedy skończony logarytm, za cenę nieścisłości (23.1).

24 Jaki problem probabilistyczny tworzy generator?

24.1 Nakładanie się klas i poziom szumu

Generator z sekcji 3 wylosował każdej klasie środek i obserwacje wokół niego. Po *ustaleniu* środków każdy punkt klasy k ma przeciętnie położenie μ_k , a typowe odchylenie *jednej cechy* wynosi 2. W 30 wymiarach dwie klasy mogą być odległe w jednych cechach, bliskie w innych. To, że klasy mają różne środki, daje modelowi sygnał; nakładający się szum oznacza, że nawet sensowna predykcja nie musi osiągnąć 100%.

Porównując warianty eksperymentu, odróżniaj s_c i s_n z (3.1). Zwiększenie s_c na ogół oddala typowe środki klas od siebie. Zwiększenie s_n poszerza chmury punktów wokół ustalonych środków. Istotna jest też ich *relacja*: jeśli przeskalujemy jednocześnie środki i szum, odległości w danych wzrosną, ale trudność rozróżnienia klas nie musi zmienić się w taki sam sposób; zmieni się natomiast skala wejść do sieci, a więc przebieg optymalizacji przy tych samych wagach i η .

24.2 Czy sieć musi mieć ReLU dla tego generatora?

To pytanie można zbadać bez rozbudowanej teorii. Jeśli dwie klasy w jednej cesze mają środki -1 i $+1$, taki sam rozrzut i równą częstość, intuicyjną granicą jest środek między nimi, tutaj 0: punkt $x > 0$ jest bliżej prawej klasy. W większej liczbie wymiarów porównanie kwadratów odległości od *ustalonych* środków μ_k i μ_l prowadzi po skróceniu wspólnego $x^\top x$ do

$$\|x - \mu_k\|^2 - \|x - \mu_l\|^2 = -2x^\top(\mu_k - \mu_l) + \|\mu_k\|^2 - \|\mu_l\|^2. \quad (24.1)$$

Granica równej odległości ma więc w x postać liniową. Przy równych licznosciach i wspólnym sferycznym szumie odpowiada to także granicom klasyfikatora, który *znałby* środki i regułę generatora. Sieć z ReLU ma większą elastyczność niż minimalnie potrzebna dla tej konstrukcji danych. Nie wynika z tego, że aktualnie *wytrenowany* model liniowy od razu osiągnie tu tę samą dokładność: trzeba by go osobno zaimplementować, wytrenować i uczciwie porównać.

24.3 Dokładność, entropia i kalibracja odpowiadają na różne pytania

Dokładność to $\text{acc} = 400^{-1} \sum_{i=1}^{400} \mathbf{1}[\hat{y}_i = y_i]$. Odpowiada na pytanie „ile razy wybraliśmy poprawną klasę?”. Cross-entropy pyta „ile udziału daliśmy prawdziwej klasie?” i mocniej reaguje na bardzo pewne pomyłki. Kalibracja pyta o związek między przydzielanym udziałem a częstością prawdziwych trafień. Obecny kod mierzy *pierwsze dwie* wielkości (choć na różnych częściach zbioru), a kalibracji nie sprawdza. Dodatkowo, bez oddzielnego testu nie należy wielokrotnie stroić hiperparametrów na tej samej walidacji i traktować najlepszego wyniku jako bezstronnej oceny uogólniania.

25 Skala danych, pochodne i zachowanie uczenia

25.1 Dlaczego wielkość wejść ma znaczenie

Wzór (15.2) mówi, że gradient $W_{1,jr}$ zawiera czynnik X_{ij} . Jeśli jedna cecha typowo ma liczby sto razy większe niż inna, jej wkład do zmian wag też może być większy, choć dokładny efekt zależy od aktywacji i innych wag. Z kolei (14.1) pokazuje, że sygnał dochodzący do pierwszej warstwy jest mnożony przez W_2 . Przy bardzo małych początkowych wagach sygnał pierwszej warstwy może być słaby, zwłaszcza gdy sieć jest głębsza. W tym projekcie dwie warstwy i skala wag 0.01 wystarczyły do zadanego eksperymentu. Nie należy jednak mechanicznie przenosić tej skali na inną liczbę warstw lub inne dane.

W języku z analizy gradient jest lokalną informacją: dla małej zmiany $\Delta\theta$ można przybliżyć $L(\theta + \Delta\theta) \approx L(\theta) + \nabla L(\theta)^\top \Delta\theta$. Wybór $\Delta\theta = -\eta \nabla L(\theta)$ daje w tym przybliżeniu zmianę $-\eta \|\nabla L(\theta)\|^2 \leq 0$. To wyjaśnia znak minus w regule uczenia. Przybliżenie nie gwarantuje spadku *rzeczywistej* straty, gdy η jest za duże albo gdy zmienia się partia między pomiarami. Nie jest to twierdzenie o monotoniczności obserwowanej walidacji.

25.2 Czego nie przenosić bezpośrednio z regresji liniowej

W regresji liniowej z MSE gradienty mają zwięzłą postać wynikającą z jednej transformacji $Xw + b$; przy standardowych założeniach problem jest wypukły. Tutaj transformacja $XW_1 \rightarrow \text{ReLU} \rightarrow W_2$ zależy od aktywnych neuronów, a cała funkcja celu nie jest ogólnie wypukła względem wszystkich wag. W praktyce nie ma jednej prostej *normal equation* rozwiązującej jednocześnie W_1 i W_2 dla tej sieci. Dlatego liczymy pochodne od straty do każdej warstwy i aktualizujemy wagi iteracyjnie. Przy *ustalonym* A_1 druga warstwa jest łatwiejsza do analizy, ale w trakcie treningu samo A_1 zmienia się razem z W_1 .

26 Audyt kodu: co działa dobrze i gdzie są granice

26.1 Decyzje trafne w projekcie edukacyjnym

Kod oddziela funkcje aktywacji i stratę od klasy modelu, oddziela wyprowadzenie gradientów od miejsca aktualizacji wag i ma niezależną kontrolę gradientów. Numerycznie stabilniejszy softmax odejmuje maksimum. Tasowanie przed każdą epoką chroni przed uczeniem zawsze w kolejności klas. Walidacja jest oddzielona od przykładów używanych przez `backward`. Te wybory ułatwiają prześledzenie rachunku i zauważenie błędów w każdej warstwie.

26.2 Uproszczenia i ograniczenia do świadomego zbadania

Miejsce	Co dokładnie oznacza i kiedy ma znaczenie
<code>layers.py</code> , <code>model.py</code>	Dodanie 10^{-9} do p w stracie i użycie $p-y$ w gradientzie daje dwie minimalnie różne funkcje; różnica staje się duża przy skrajnie małym p_c (sekcja 23). Stabilna strata liczona z logitów mogłaby usunąć tę niespójność.
<code>relu_backward</code>	Kod przepuszcza sygnał przy $z = 0$. W tym miejscu ReLU nie ma zwykłej pochodnej. Różnica skończona może pokazać inny wynik, gdy test trafi dokładnie w granicę.

Miejsce	Co dokładnie oznacza i kiedy ma znaczenie
<code>gradient_check</code>	Dla dwóch zerowych gradientów mianownik błędu względnego jest 0; podczas wyjątku modyfikowana waga może nie wrócić do oryginału. W zwykłym pomyślnym teście z repozytorium oba problemy nie wystąpiły.
<code>train.py</code>	Średnia strat partii miesza kolejne stany wag; nie jest stratą całego zbioru z końca epoki. Przy niepełnej ostatniej partii jej wpływ w średniej byłby zbyt duży.
<code>run_experiments.py</code>	Liczba 10 jest wpisana na stałe przy kodowaniu one-hot i w konstruktorze. Zmiana liczby klas w generatorze wymaga synchronizacji tych miejsc. Podział na trening i walidację nie jest stratyfikowany.
Losowanie w kilku plikach	Wywołania <code>np.random.seed</code> zmieniają globalny generator. Osobne <code>np.random.default_rng(seed)</code> dla danych, wag i tasowania rozdzieliłyby źródła losowości.
<code>visualize.py</code>	X osi jest indeksem listy od 0, choć czytamy go jako numer epoki od 1; PNG jest nadpisywany, a figura nie jest zamykana. Nie zmienia to wyników pojedynczego treningu.
Cały projekt	Brak jawnego sprawdzania kształtów, pustych tablic, <code>nan/inf</code> i dodatniości hiperparametrów. Poprawne wejście w tym eksperymencie spełnia oczekiwania, lecz błędne wejście może powodować niejasne błędy NumPy.

Brak automatycznego różniczkowania, regularizacji, Adama, funkcji zapisywania wag i wielu warstw jest tu *uproszczeniem celu edukacyjnego*; nie należy nazywać tego błędem. Natomiast niespójność straty i gradientu jest konkretną niedokładnością, której sens matematyczny można wyprowadzić. To rozróżnienie pomaga przy recenzowaniu kodu: najpierw stwierdzamy, co *naprawdę* jest implementowane, potem decydujemy, czy poprawka odpowiada celowi ćwiczenia.

27 Co badać dalej w samym projekcie

27.1 Trzy kontrolowane zmiany

Pierwszy eksperyment: zmień `noise_scale` z 2 na 1, pozostawiając inne hiperparametry. Hipoteza: przy mniej nakładających się chmurach dokładność zwykle rośnie. Zrób kilka niezależnych losowań i zapisz średnią oraz rozrzut; jedno uruchomienie nie wystarczy. Drugi: porównaj $h = 4, 64$ i 256 , pilnując, żeby pozostałe warunki były takie same. Hipoteza nie jest monotoniczna przy 30 epokach, bo oprócz pojemności zmienia się przebieg uczenia. Trzeci: zmień `lr` na $0.001, 0.01, 0.1$ i porównaj stratę *na tym samym zbiorze po epoce*, nie tylko istniejącą średnią z partii. Dla dużego kroku może pojawić się niestabilność; nie zakładaj tego bez pomiaru.

27.2 Most do znanej regresji liniowej

Usuń dla eksperymentu warstwę ukrytą i pozostaw $Z = XW + b$, softmax, cross-entropy. Otrzymasz regresję logistyczną wieloklasową, mimo że nazwa zawiera słowo „regresja”: celem jest klasyfikacja. Gradient wag tego prostszego modelu ma tę samą logikę co (13.2): $\nabla_W L = X^T(P - Y)/B$. Zauważ, co się zmienia względem regresji liniowej z MSE: *sygnał wyjściowy* to $P - Y$ zamiast sygnału proporcjonalnego do $\hat{y} - y$, a sam mechanizm „wejście razy sygnał,

suma po punktach” pozostaje. Dzięki temu materiał o gradientzie regresji liniowej przenosi się na pierwszy etap backpropagation, lecz nie zastępuje bramki ReLU i drugiej warstwy.

27.3 Regularizacja jako możliwy dalszy krok

W tym repozytorium *nie* ma kary za duże wagi. Jeśli w przyszłości dodasz karę typu Ridge do funkcji celu, jedna możliwa definicja to

$$L_\lambda = L + \frac{\lambda}{2}(\|W_1\|_F^2 + \|W_2\|_F^2), \quad \lambda \geq 0. \quad (27.1)$$

Z zasady różniczkowania kwadratu gradient *dodatkowej kary* dla W_1 to λW_1 , a dla W_2 to λW_2 ; biasów nie uwzględniono w tej konkretnej definicji. *Lambda* nie opisuje obecnych wyników: to opcjonalne rozszerzenie pokazujące, jak znana z regresji kara weszłaby do dwóch tablic gradientów. Po dodaniu jej do kodu musiałbyś pamiętać, by `gradient_check` porównywał gradient z *tą samą nową stratą*, inaczej test zgłosi rozbieżność mimo poprawnej implementacji poszczególnych fragmentów.

28 Pytania kontrolne i ćwiczenia

28.1 Wyjaśnij kod bez patrzenia na wzory

1. Dlaczego `forward` zwraca także `cache`? Który zapisany element jest potrzebny do gradientu W_2 , a który do maski ReLU?
2. Jaki kształt ma `d_logits` dla partii 16 punktów? Dlaczego `a1.T @ d_logits` ma kształt tablicy W_2 ?
3. Dlaczego `params[name] -= ...` zmienia model, choć `params()` zwraca nowy słownik?
4. Wyjaśnij różnicę między `train_loss[-1]` i stratą całego `x_train` zmierzoną po epoce. Jak byś dopisał drugi pomiar?
5. Co w `main` trzeba zmienić, żeby uruchomić generator z pięcioma klasami i pozostawić zgodne kształty?

28.2 Zrób krótki rachunek

1. Dla $z = (0, 0, 0)$ wyznacz softmax i stratę, jeżeli poprawna jest klasa 2. Jak zmienia się strata, gdy podniesiesz wyłącznie jej logit? Uzasadnij znaki gradientu bez mnożenia macierzy.
2. Dla $x = (2, 1)$ oraz W_1 z (8.2) policz nowe z_1 i wskaż, czy oba neurony przepuszczają gradient. Następnie sprawdź `forward` w Pythonie.
3. Dla dwóch punktów z sygnałami $D_{2,0}^{(1)} = -0.1$, $D_{2,0}^{(2)} = 0.05$ i aktywacjami $a_{1,0}^{(1)} = 0.5$, $a_{1,0}^{(2)} = 0.2$ wyprowadź $\partial L / \partial W_{2,00}$ i porównaj z małą tablicą NumPy.
4. Dla $z_1 = (-1, 2)$ i $D_{A_1} = (0.4, -0.3)$ podaj wynik `relu_backward`. Który neuron wnosi coś do ∇W_1 ?
5. Dla jednej wagi $w = 0.4$, gradientu -0.217682 i $\eta = 0.1$ policz nową wagę. Co zrobiłaby zmiana η na 10 i dlaczego lokalna pochodna nie wystarcza wtedy do obietnicy poprawy?

28.3 Praktyczny test pełnego zrozumienia

Bez otwierania opracowania opowiedz: skąd bierze się `probs`; dlaczego strata używa logarytmu udziału poprawnej klasy; czemu `d_logits` ma postać $(\text{probs} - y)/B$; gdzie występuje suma po *klasach*, a gdzie po *obserwacjach*; którą pojedynczą linijką pętla zmienia model. Jeśli potrafisz też przewidzieć kształt każdej tablicy i wyjaśnić, czemu przebieg na walidacji nie jest tym samym co kontrola gradientu, masz solidną podstawę do następnego projektu.

29 Słownik potrzebny do powrotu do kodu

Termin	Znaczenie w tym projekcie
<i>Bias</i> – przesunięcie	Stała dodawana do sumy ważonej; <code>b1</code> , <code>b2</code> .
<i>Batch</i> – partia	Punkty użyte do jednego wyliczenia średniego gradientu; zwykle 32.
<i>Broadcasting</i> – rozgłaszanie	Zastosowanie wektora biasu do każdego wiersza tablicy bez jawnego powielania w kodzie.
<i>Cache</i> – pamięć wyników	Słownik wartości pośrednich <code>forward</code> , potrzebnych do <code>backward</code> .
<i>Cross-entropy</i> – entropia krzyżowa	Uśredniona kara za mały udział przypisany prawdziwej klasie.
<i>Epoch</i> – epoka	Jedno przejście przez 1600 punktów treningowych, tutaj 50 kroków.
<i>Forward pass</i> – przebieg w przód	Liczenie kolejnych tablic Z_1, A_1, Z_2, P przy bieżących wagach.
<i>Gradient</i>	Zbiór pochodnych straty względem uczonych parametrów; wskazuje lokalny wzrost straty.
<i>Gradient check</i> – kontrola gradientu	Porównanie <code>backward</code> z dwoma pomiarami straty po zmianie jednej wagi.
<i>Learning rate</i> – współczynnik uczenia	Czynnik η skalujący odejmowany gradient; w <code>main</code> wynosi 0.01.
<i>Logit</i> – surowa ocena	Wynik drugiej warstwy <i>przed</i> softmax; <code>z2</code> .
<i>One-hot</i> – pojedyncza jedynka	Etykieta kodowana zerami i jedną jedynką dla poprawnej klasy.
<i>ReLU</i>	Funkcja $\max(0, z)$; zeruje ujemne preaktywacje neuronów.
<i>Softmax</i>	Zamienia wektor logitów na dodatnie udziały sumujące się do 1.
<i>Validation</i> – walidacja	400 punktów bez aktualizacji wag, użytych do pomiaru <i>accuracy</i> po epokach.

30 Końcowa synteza

Jedna obserwacja przechodzi przez dwa iloczyny macierzy, ReLU i softmax. Entropia krzyżowa ocenia udział poprawnej klasy. Pochodne zaczynają się od $P - Y$, następnie przez aktywacje, wagi drugiej warstwy i bramkę ReLU docierają do pierwszej warstwy. Każdy iloczyn macierzy w `backward` liczy sumy wkładów, które potrafisz również zapisać dla jednej wagi. `train` odejmuje gradient od czterech tablic, a `main` sprawdza rachunek numerycznie i mierzy zachowanie na oddzielnych 400 punktach. To jest matematyczny i implementacyjny obraz *tego konkretnego projektu*; aneks pozwala wrócić do każdej omawianej funkcji w jej oryginalnym kontekście.

31 Niezmieniony kod źródłowy

Poniżej znajduje się cały kod sześciu plików z rewizji 32c0899c48ad repozytorium projektu. Zawartość plików nie została przerobiona na nową implementację. W aneksie liczby przy lewym brzegu oznaczają *linię plików*; łamanie zbyt długiego wiersza wynika tylko ze składu strony.

Plik	Gdzie został objaśniony
run_experiments.py	Sekcje 3, 4 i 21.
layers.py	Sekcje 6, 9 i 10.
model.py	Sekcje 7, 8, 12–15, 19, 20.
train.py	Sekcja 18.
visualize.py	Sekcja 22.
__init__.py	Sekcje 1 i 22; plik pusty.

__init__.py

Plik ma zero bajtów i nie zawiera żadnej linii kodu.

layers.py

```

1 import numpy as np
2
3
4 def relu(x: np.ndarray) -> np.ndarray:
5     return np.maximum(x, 0)
6
7
8 def relu_backward(dout: np.ndarray, x: np.ndarray) -> np.ndarray:
9     return np.where(x >= 0, dout, 0)
10
11
12 def softmax(logits: np.ndarray) -> np.ndarray:
13     logits_shifted = logits - logits.max(axis=-1, keepdims=True)
14     exp_z = np.exp(logits_shifted)
15     return exp_z / exp_z.sum(axis=-1, keepdims=True)
16
17
18 def cross_entropy(y_true_onehot: np.ndarray, probs: np.ndarray) -> np.ndarray:
19     eps = 1e-9
20     per_sample = -np.sum(y_true_onehot * np.log(probs + eps), axis=1)
21     return np.mean(per_sample)

```

model.py

```

1 import numpy as np
2 from .layers import relu, relu_backward, softmax, cross_entropy
3
4
5 class TwoLayerNet:
6     def __init__(
7         self, n_features: int, n_hidden: int, n_classes: int, seed: int = 42
8     ) -> None:
9         np.random.seed(seed)
10         self.W1 = np.random.randn(n_features, n_hidden) * 0.01
11         self.W2 = np.random.randn(n_hidden, n_classes) * 0.01
12         self.b1 = np.zeros(shape=(n_hidden,))
13         self.b2 = np.zeros(shape=(n_classes,))
14

```

```

15     def forward(self, X: np.ndarray) -> tuple[np.ndarray, dict]:
16         z1 = X @ self.W1 + self.b1
17         a1 = relu(z1)
18         z2 = a1 @ self.W2 + self.b2
19         probs = softmax(z2)
20         cache = {"X": X, "z1": z1, "a1": a1, "z2": z2, "probs": probs}
21
22         return probs, cache
23
24     def backward(self, y_true_onehot: np.ndarray, cache: dict) -> dict:
25         n = y_true_onehot.shape[0]
26         d_logits = (cache["probs"] - y_true_onehot) / n
27         dW2 = cache["a1"].T @ d_logits
28         db2 = d_logits.sum(axis=0)
29         d_a1 = d_logits @ self.W2.T
30         d_z1 = relu_backward(d_a1, cache["z1"])
31         dW1 = cache["X"].T @ d_z1
32         db1 = d_z1.sum(axis=0)
33         return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}
34
35     def predict(self, X: np.ndarray) -> np.ndarray:
36         probs = self.forward(X)[0]
37         return probs.argmax(axis=1)
38
39     def params(self) -> dict:
40         return {"W1": self.W1, "b1": self.b1, "W2": self.W2, "b2": self.b2}
41
42     def gradient_check(
43         self, X: np.ndarray, y_true_onehot: np.ndarray, eps: float = 1e-5
44     ) -> dict:
45         cache = self.forward(X)[1]
46         analytic_grads = self.backward(y_true_onehot, cache)
47
48         relative_errors = {}
49
50         for key in analytic_grads.keys():
51             param = getattr(self, key)
52             numeric = np.zeros_like(param)
53
54             it = np.nditer(param, flags=["multi_index"])
55             for _ in it:
56                 idx = it.multi_index
57                 original = param[idx]
58                 param[idx] = original + eps
59                 loss_plus = cross_entropy(y_true_onehot, self.forward(X)[0])
60                 param[idx] = original - eps
61                 loss_minus = cross_entropy(y_true_onehot, self.forward(X)[0])
62                 param[idx] = original
63                 numeric[idx] = (loss_plus - loss_minus) / (2 * eps)
64
65             analytic = analytic_grads[key]
66             relative_errors[key] = np.linalg.norm(numeric - analytic) / (
67                 np.linalg.norm(numeric) + np.linalg.norm(analytic)
68             )
69
70         return relative_errors

```

run_experiments.py

```

1 import numpy as np
2 from .model import TwoLayerNet
3 from .layers import cross_entropy
4 from .train import train
5 from .visualize import plot_training_curves

```

```

6
7
8 def generate_synthetic_data(
9     n_classes: int = 10,
10    n_features: int = 30,
11    n_samples_per_class: int = 200,
12    center_scale: float = 1.0,
13    noise_scale: float = 2.0,
14    seed: int = 0,
15 ) -> tuple[np.ndarray, np.ndarray]:
16     np.random.seed(seed)
17
18     X_list = []
19     y_list = []
20     for class_idx in range(n_classes):
21         center = np.random.randn(n_features) * center_scale
22         points = center + np.random.randn(n_samples_per_class, n_features) * noise_scale
23         labels = np.full(n_samples_per_class, class_idx)
24
25         X_list.append(points)
26         y_list.append(labels)
27
28     X = np.vstack(X_list)
29     y = np.concatenate(y_list)
30
31     perm = np.random.permutation(len(X))
32     X = X[perm]
33     y = y[perm]
34
35     return X, y
36
37
38 def main() -> None:
39     X, y = generate_synthetic_data()
40     split = int(0.8 * len(X))
41     X_train, X_val = X[:split], X[split:]
42     y_train, y_val = y[:split], y[split:]
43
44     y_train_onehot = np.zeros((len(y_train), 10))
45     y_train_onehot[np.arange(len(y_train)), y_train] = 1
46
47     model = TwoLayerNet(30, 64, 10)
48     probs, cache = model.forward(X_train)
49     loss = cross_entropy(y_train_onehot, probs)
50     assert np.isclose(loss, np.log(10), atol=0.05)
51
52     grad_errors = model.gradient_check(X_train[:16], y_train_onehot[:16])
53     assert all(err < 1e-4 for err in grad_errors.values())
54
55     history = train(model, X_train, y_train_onehot, X_val, y_val, lr=0.01, n_epochs=30)
56     assert history["val_accuracy"][-1] > 0.6
57     plot_training_curves(history["train_loss"], history["val_accuracy"])
58
59
60 if __name__ == "__main__":
61     main()

```

train.py

```

1 import numpy as np
2 from .model import TwoLayerNet
3 from .layers import cross_entropy
4
5

```

```

6 def train(
7     model: "TwoLayerNet",
8     X_train: np.ndarray,
9     y_train_onehot: np.ndarray,
10    X_val: np.ndarray,
11    y_val: np.ndarray,
12    lr: float = 0.5,
13    batch_size: int = 32,
14    n_epochs: int = 30,
15 ) -> dict:
16     train_loss = []
17     val_accuracy = []
18
19     for epoch in range(n_epochs):
20         batch_loss = []
21         indices = np.random.permutation(X_train.shape[0])
22         X_shuffled = X_train[indices]
23         y_shuffled = y_train_onehot[indices]
24         for batch in range(0, X_train.shape[0], batch_size):
25             X_batch = X_shuffled[batch : batch + batch_size]
26             y_batch = y_shuffled[batch : batch + batch_size]
27             probs, cache = model.forward(X_batch)
28             grads = model.backward(y_batch, cache)
29             params = model.params()
30             for name in params:
31                 params[name] -= lr * grads[name]
32
33             batch_loss.append(cross_entropy(y_batch, probs))
34         train_loss.append(np.mean(batch_loss))
35
36         preds = model.predict(X_val)
37         val_accuracy.append((preds == y_val).mean())
38
39     return {"train_loss": train_loss, "val_accuracy": val_accuracy}

```

visualize.py

```

1 from pathlib import Path
2 import matplotlib.pyplot as plt
3
4 FIGURES_DIR = Path(__file__).parent / "figures"
5
6
7 def plot_training_curves(train_loss: list[float], val_accuracy: list[float]) -> None:
8     FIGURES_DIR.mkdir(exist_ok=True)
9     fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, sharex=True)
10    ax1.plot(train_loss)
11    ax1.set_title("Train loss")
12    ax1.set_ylabel("loss (cross-entropy)")
13
14    ax2.plot(val_accuracy)
15    ax2.set_title("Val accuracy")
16    ax2.set_xlabel("epoch")
17    ax2.set_ylabel("accuracy")
18    fig.savefig(FIGURES_DIR / "training_curves.png")

```